Distributed Computation Offloading System

Project overview, current implementation and proposed next steps

Abstract

This project develops a distributed computation offloading platform that allows a client to send a compute task to a server instead of running it locally. The server authenticates the request, places the work on a Redis-backed Celery queue, and assigns it to an available worker. Workers run the submitted workload inside task-specific Docker containers with execution time and resource limits. The client can retrieve the result, while the web dashboard reports queue and worker activity. The current prototype supports general Python scripts, NumPy-based numeric computation, PyTorch inference, FFmpeg rendering, and Python, JavaScript, and C++ execution through a multi-language runner. It also includes user login, task history, downloadable render artifacts, and CSV/PDF reports. The next project phase should focus on repeatable evaluation under concurrent load, stronger operational security and credential handling, and documentation of measured results. Adaptive scheduling and deployment across multiple machines are proposed extensions, subject to project priorities and available time.

Project purpose and workflow

The project explores how compute-intensive work can be moved from a client to a pool of server-side workers. Its central engineering problem is to accept different kinds of work through one interface, execute that work asynchronously and in isolation, then return useful results with visibility into system activity.

1. **1. Submit** A user signs in through the web interface or a client script and submits source code or a task payload with a task type.
2. **2. Authenticate and queue** FastAPI validates the bearer token and request. Celery publishes the task to Redis and immediately returns a task ID, so submission does not wait for execution.
3. **3. Select and execute** An available Celery worker takes the queued task and launches the matching Docker runner. The runner has bounded CPU, memory and time; network access is disabled and the container filesystem is read-only except for its temporary work area.
4. **4. Collect the result** The worker captures standard output, errors, execution time and worker identity. Render tasks can also save output files as downloadable artifacts.
5. **5. View and record** The client polls for completion. The dashboard periodically refreshes queue depth, worker status and recent tasks. SQLite stores user accounts and task records; authenticated users can download CSV or PDF reports for their own records.

System components

Component	Responsibility
Client and web pages	Login, task submission, result polling and a live-refreshing worker/task dashboard.
FastAPI service	Login and token checks, task submission, result and artifact retrieval, dashboard data and report endpoints.
Redis and Celery	Redis holds queued work and task results; Celery workers consume tasks asynchronously. Multiple workers can share the queue.
Docker runners	Separate images isolate general Python, numeric, ML, rendering and multi-language workloads.

SQLite	Stores account hashes and task metadata, status, errors and execution time.

What is implemented

- Asynchronous task submission and result retrieval through FastAPI, Celery and Redis, with multiple worker processes able to consume the shared queue.
- Five task routes: general Python, NumPy numeric work, PyTorch inference, FFmpeg rendering, and multi-language execution for Python, JavaScript and C++.
- Docker-based execution controls, including per-run CPU and memory limits, timeouts, disabled container networking, dropped Linux capabilities, a read-only container root and a bounded temporary filesystem.
- JWT-based sign-in with hashed passwords, per-IP task-submission rate limiting, and ownership checks on task results and render downloads. CSV/PDF reports are scoped to the signed-in user.
- A web task-submission page and dashboard with queue depth, worker busy/idle state and recent task status; render outputs are retained as downloadable artifacts.

Current scope and work still needed

The current implementation is a functional prototype. Its worker pool can distribute work across multiple worker processes sharing Redis, and concurrent-client/load-test scripts are present. The repository does not yet provide a completed, documented performance evaluation demonstrating throughput, latency, scaling behaviour or recovery under failure. Current deployments and examples are oriented around one development machine; multi-machine deployment and adaptive scheduling are not established capabilities.

The web client checks for results by polling rather than receiving push notifications. Resource ceilings and timeouts are fixed in the worker code. Sample client scripts contain hard-coded development credentials and should be updated to read credentials safely before they are used beyond a local demonstration. The local auto-generated JWT key simplifies development; a deployed environment needs a securely managed, consistent key.

Proposed next steps

Priority	Work	Expected outcome
1. Stabilize and secure the demo	Remove sample credentials from client scripts, use environment-based configuration, confirm account setup instructions, and document the local run path and supported task types.	A repeatable demo that does not rely on committed passwords.
2. Build an evaluation plan	Define workload mixes and measurements such as queue wait, end-to-end latency, throughput, success rate, worker utilization and recovery after a worker interruption.	A test matrix and reproducible baseline results.
3. Run controlled concurrency tests	Use the existing multi-client workload scripts, increase load in measured steps, record results and compare one worker with multiple workers.	Evidence of how the current queue and worker pool behave under load.
4. Address reliability and operations	Review task retry and timeout behaviour, worker health visibility, cleanup/retention of render artifacts and result records, and deployment configuration.	A documented operational and failure-handling model.
5. Decide on stretch features	After baseline results and guide feedback, prioritize either adaptive worker-aware scheduling, multi-machine deployment, or push-based result updates. These are proposed extensions, not completed features.	A scoped enhancement plan tied to project requirements and remaining time.

6. Complete academic documentation	Produce architecture and sequence diagrams, test cases, measured-result charts, limitations, and a final user/demo guide.	A project report that explains design choices and supports claims with evidence.